Ryerson University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	W2026

Instructor:	Muhammad Jaseemuddin

Assignment/Lab Number:	4-5
Assignment/Lab Title:	FastAPI and Containers

Submission Date:	02/04/2026
Due Date:	03/04/2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Meng	Alex	501095337	07	A.M

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

Introduction:

The goal of this lab is to simulate the RESTful communication of an operator with a server that simulates the mine defusing rovers from previous labs. The server will implement the following functions:

Map		
Route	**Method**	**Description**
/map	GET	To retrieve the 2D array of the field.
/map	PUT	To update the height and width of the field.
Mines		
Route	**Method**	**Description**
/mines	GET	To retrieve the list of all mines, the response should include the serial number of the mines, and coordinates.
/mines/:id	GET	To retrieve a mine with the ":id", the response should include the serial number of the mine, and coordinates.
/mines/:id	DELETE	To delete a mine with the ":id"
/mines	POST	To create a mine. The coordinates (X and Y), along with the serial number should be required in the body of the request. The ID of the mine should be returned in the response upon successful creation.
/mines/:id	PUT	To update a mine. The coordinates (X and Y), along with the serial number should be optional in the body of the request. Only the included parameters should get updated upon receiving the request. The response must include the full updated mine object.

Rover		
Route	**Method**	**Description**
/rovers	GET	To retrieve the list of all rovers, the response should at least include the ID and rover status ("Not Started", "Finished", "Moving", or "Eliminated").
/rovers/:id	GET	To retrieve a rover with the ":id", the response should include the ID, status ("Not Started", "Finished", "Moving", or "Eliminated"), latest position and list of commands of the rover.
/rovers	POST	To create a rover. The list of commands as a String should be required in the body of the request. The ID of the rover should be returned in the response upon successful creation.
/rovers/:id	DELETE	To delete a rover with the ":id"
/rovers/:id	PUT	To send the list of commands to a rover as a String. Note if the rover status is not "Not Started" nor "Finished", a failure response should be returned.
/rovers/:id/dispatch	POST	To dispatch a rover with the ":id", the response should include the ID, status, latest position and list of the executed commands of the rover.

Part 1

The server was implemented in Python with FastAPI. All of the endpoints in the manual were set up as specified. The dispatch rover command starts a new thread running the rover's functions. Both the server and dispatched rovers use locks to control access to shared resources like the map array. The code is shown below:

```
from fastapi import FastAPI
from pydantic import BaseModel
import json
import uvicorn
import roverClient
from threading import Lock, Thread

Map = []
rovers = {"rovers": []}
mines = {"mines": []}
```

```python
mapLock = Lock()
roverLock = Lock()
maxMineID = 0
maxRoverID = 0
height = 0
width = 0


class createMineReq(BaseModel):
    x : int
    y : int
    serial : str

class updateMineReq(BaseModel):
    x : int | None
    y : int | None
    serial : str | None
    armed : bool | None
    pin : str | None

class createRoverReq(BaseModel):
    cmds : str

class updateCmdReq(BaseModel):
    cmds : str

class mapUpdateReq(BaseModel):
    height: int | None
    width: int | None


server = FastAPI()

# Map
@server.get("/map")
def getMap():
    return {"map" : Map}
```

```python
@server.put("/map/")
def putMap(body: mapUpdateReq):
    global Map, height, width, mapLock
    # add extra rows
    with mapLock:
        if height <= body.height:
            for i in range(body.height - height):
                row = []
                for j in range(body.width):
                    row.append(0)
                Map.append(row)
        else:
            # remove extra rows from Map
            for i in range(height - body.height):
                Map.pop(len(Map)-(i+1))
        # add cells to existing rows
        if width <= body.width:
            for i in range(height):
                for j in range(body.width - width):
                    Map[i].append(0)
        else:
            # remove extra cells from rows
            for i in range(body.height):
                for j in range(width - body.width):
                    Map[i].pop(len(Map[i])-(j+1))
        height = body.height
        width = body.width
        for mine in mines["mines"]:
            Map[mine["y"]][mine["x"]] = mine["id"]
        return {"height": height, "width": width, "map": Map}

# Mines
@server.get("/mines")
def getMines():
    return mines

@server.get("/mines/{id}")
def getMineID(id: int):
```

```
    for mine in mines["mines"]:
        if mine["id"] == id:
            return {"Success": True, "mine": mine, "error": None}
    return {"Success": False, "mine": None, "error": f"Error: mine with ID {id}
not found"}

@server.delete("/mines/{id}")
def delMineID(id: int):
    global mines, mapLock
    for mine in mines["mines"]:
        if mine["id"] == id:
            with mapLock:
                Map[mine["y"]][mine["x"]] = 0
            mines["mines"].remove(mine)
            return {"Success": True, "error": None}
    return {"Success": False, "error": f"Error: mine with ID {id} not found"}

@server.post("/mines/")
def createMine(body: createMineReq):
    global maxMineID, mines, mapLock
    maxMineID += 1
    if Map[body.y][body.x] == 0:
        mines["mines"].append({"id": maxMineID, "Serial Number": body.serial,
"x": body.x, "y": body.y, "armed": True, "pin": ""})
        with mapLock:
            Map[body.y][body.x] = maxMineID
        return {"Success": True, "id": maxMineID, "error": None}
    return {"Success": False, "id": None, "error": f"Error: mine already exists
at {body.x}, {body.y}"}

@server.put("/mines/{id}/")
def updateMine(id: int, body: updateMineReq):
    global mines, Map
    for mine in mines["mines"]:
        if mine["id"] == id:
            if body.serial != None:
                mine["Serial Number"] = body.serial
            if body.x != None:
```

```
                with mapLock:
                    Map[mine["y"]][mine["x"]] = 0
                    mine["x"] = body.x
                    Map[body.y][body.x] = mine["id"]
            if body.y != None:
                with mapLock:
                    Map[mine["y"]][mine["x"]] = 0
                    mine["y"] = body.y
                    Map[body.y][body.x] = mine["id"]
            if body.armed != None:
                mine["armed"] = body.armed
            if body.pin != None:
                mine["pin"] = body.pin

            return {"Success": True, "mine": mine, "error": None}
    return {"Success": False, "mine": None, "error": f"Error: mine with ID {id}
not found"}

# Rovers
@server.get("/rovers")
def getRovers():
    return rovers

@server.get("/rovers/{id}")
def getRoverID(id: int):
    for rover in rovers["rovers"]:
        if rover["id"] == id:
            return {"Success": True, "rover": rover, "error": None}
    return {"Success": False, "rover": None, "error": f"Error: rover with ID
{id} not found"}

@server.post("/rovers/")
def createRover(roverReq: createRoverReq):
    global rovers, maxRoverID
    print(roverReq.cmds)
    with roverLock:
        maxRoverID += 1
        rovers["rovers"].append({"id": maxRoverID, "status": "Not Started",
```

```
"commands": roverReq.cmds, "x": 0, "y": 0})
        return {"Success": True, "id": maxRoverID}

@server.delete("/rovers/{id}")
def delRover(id: int):
    global roverLock, rovers
    with roverLock:
        for rover in rovers["rovers"]:
            if rover["id"] == id:
                rovers["rovers"].remove(rover)
                return {"Success": True, "error": None}
        return {"Success": False, "error": f"Error: rover with ID {id} not
found"}


@server.put("/rovers/{id}/")
def sendCmd(id: int, cmds: updateCmdReq):
    global rovers
    for rover in rovers["rovers"]:
        if rover["id"] == id:
            with roverLock:
                if rover["status"] == "Not started" or "Finished":
                    rover["commands"] = cmds.cmds
                    return {"Success": True, "error": None}
                else:
                    return {"Success": False, "error": f"Error: rover status is
{rover["status"]}"}
    return {"Success": False, "error": f"Error: rover with ID {id} not found"}


@server.post("/rovers/{id}/dispatch/")
async def dispatchRover(id: int):
    global rovers, Map, height, width, mapLock, roverLock
    for rover in rovers["rovers"]:
        if rover["id"] == id:
            if rover["status"] == "Not Started":
                rover["status"] = "Moving"
                # insert rover code here
```

```
                roverThread = Thread(target=roverClient.run, args=(Map, height,
width, rover, mapLock, roverLock))
                roverThread.start()

                return {"Success": True, "rover": rover, "error": None}
            else:
                return {"Success": False, "error": f"Error: rover in
{rover["status"]} state"}
    return {"Success": False, "error": f"Error: rover with ID {id} not found"}

@server.on_event("startup")
def startup():
    global height, width, Map, mines, maxMineID
    Map = [[0,0,0],[0,0,0],[0,0,0],[0,0,0]]
    height = 4
    width = 3

if __name__ == '__main__':
    uvicorn.run("server:server", host="0.0.0.0", port=8000, reload=True)
```

Part 2

The operator program is implemented in Python as a text-based UI, as seen below:

```
    [0, 0, 0]
    [0, 1, 0]
    [0, 0, 0]
    [0, 0, 0]

----OPERATOR----

Spawn Rover: 1
See Rovers: 2
Edit Rover Commands: 3
Delete Rover: 4
Dispatch Rover: 5
Edit Map: 6
Create Mine: 7
Edit Mine: 8
See Mine Details: 9
Delete Mine: 10
Exit: -1
>>
```

The operator program prompts the user for input to perform various actions. These actions are implemented by calling the appropriate request from the server. The functions should be self-explanatory based on the prompts.

Spawn rover:

```
Spawn Rover
Commands: (L, R, M, D)
>> MMD
Rover successfully spawned with ID 3
Press enter to return to main menu
```

See (All) Rovers:

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> -1
{'id': 1, 'status': 'Not Started', 'commands': 'M', 'x': 0, 'y': 0}
{'id': 2, 'status': 'Finished', 'commands': 'M', 'x': 0, 'y': 1}
{'id': 3, 'status': 'Not Started', 'commands': 'MMD', 'x': 0, 'y': 0}
Press enter to return to main menu
```

See (1) Rover:

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> 1
{'id': 1, 'status': 'Not Started', 'commands': 'M', 'x': 0, 'y': 0}
Press enter to return to main menu
```

Edit Rover Commands:

```
Update Rover Commands
Rover ID:
>> 1
Commands: (L, R, M, D)
>> MMLM
Commands updated
Press enter to return to main menu
```

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> 1
{'id': 1, 'status': 'Not Started', 'commands': 'MMLM', 'x': 0, 'y': 0}
Press enter to return to main menu
```

Delete Rover:

```
Delete Rover
Rover ID:
>> 3
Rover 3 successfully deleted
Press enter to return to main menu
```

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> -1
{'id': 1, 'status': 'Not Started', 'commands': 'MMLM', 'x': 0, 'y': 0}
{'id': 2, 'status': 'Finished', 'commands': 'M', 'x': 0, 'y': 1}
Press enter to return to main menu
```

Dispatch Rover:

```
Dispatch Rover
Rover ID:
>> 1
Successfully dispatched Rover 1
{'id': 1, 'status': 'Moving', 'commands': 'MMLM', 'x': 1, 'y': 2}
Press enter to return to main menu
```

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> -1
{'id': 1, 'status': 'Finished', 'commands': 'MMLM', 'x': 1, 'y': 2}
{'id': 2, 'status': 'Finished', 'commands': 'M', 'x': 0, 'y': 1}
Press enter to return to main menu
```

Edit Map:

```
Edit Map
New Height:
>> 5
New Width:
>> 5
Press enter to return to main menu
```

```
    [0, 0, 0, 0, 0]
    [0, 1, 0, 0, 0]
    [0, 0, 0, 0, 0]
    [0, 0, 0, 0, 0]
    [0, 0, 0, 0, 0]

----OPERATOR----

Spawn Rover: 1
See Rovers: 2
Edit Rover Commands: 3
Delete Rover: 4
Dispatch Rover: 5
Edit Map: 6
Create Mine: 7
Edit Mine: 8
See Mine Details: 9
Delete Mine: 10
Exit: -1
>>
```

Create Mine:

```
Create Mine
x position:
>> 2
y position:
>> 2
Serial Number:
>> weioursdklf
Mine successfully created at 2, 2 with ID 2
Press enter to return to main menu
```

```
[0, 0, 0, 0, 0]
[0, 1, 0, 0, 0]
[0, 0, 2, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

Edit Mine:

```
Edit Mine
Mine ID:
>> 2
x position:
>> 0
y position:
>> 0
Serial Number:
>>
Mine successfully edited:
{'id': 2, 'Serial Number': 'weioursdklf', 'x': 0, 'y': 0, 'armed': True, 'pin': ''}
Press enter to return to main menu
```

```
[2, 0, 0, 0, 0]
[0, 1, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

See (All) Mine Details:

```
See Mine Details
Mine ID: (Enter -1 to see all mines)
>> -1
{'id': 1, 'Serial Number': 'lasdkjf', 'x': 1, 'y': 1, 'armed': True, 'pin': ''}
{'id': 2, 'Serial Number': 'weioursdklf', 'x': 0, 'y': 0, 'armed': True, 'pin': ''}
Press enter to return to main menu
```

See Mine 1 Details:

```
See Mine Details
Mine ID: (Enter -1 to see all mines)
>> 1
{'id': 1, 'Serial Number': 'lasdkjf', 'x': 1, 'y': 1, 'armed': True, 'pin': ''}
Press enter to return to main menu
```

Delete Mine:

```
Delete Mine
Mine ID:
>> 2
Successfully deleted Mine 2
Press enter to return to main menu
```

Exploded Rover:

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> 4
{'id': 4, 'status': 'Not Started', 'commands': 'LMRMM', 'x': 0, 'y': 0}
Press enter to return to main menu
```

```
[0, 0, 0, 0, 0]
[0, 1, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

```
Dispatch Rover
Rover ID:
>> 4
Successfully dispatched Rover 4
{'id': 4, 'status': 'Eliminated', 'commands': 'LMRMM', 'x': 1, 'y': 2}
Press enter to return to main menu
```

```
See Mine Details
Mine ID: (Enter -1 to see all mines)
>> -1
{'id': 1, 'Serial Number': 'lasdkjf', 'x': 1, 'y': 1, 'armed': True, 'pin': ''}
Press enter to return to main menu
```

Disarmed Mine:

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> 5
{'id': 5, 'status': 'Not Started', 'commands': 'MLMDM', 'x': 0, 'y': 0}
Press enter to return to main menu
```

```
Dispatch Rover
Rover ID:
>> 5
Successfully dispatched Rover 5
{'id': 5, 'status': 'Moving', 'commands': 'MLMDM', 'x': 1, 'y': 1}
Press enter to return to main menu
```

```
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

```
See Rovers
Rover ID: (Enter -1 to see all rovers)
>> 5
{'id': 5, 'status': 'Finished', 'commands': 'MLMDM', 'x': 2, 'y': 1}
Press enter to return to main menu
```

```
See Mine Details
Mine ID: (Enter -1 to see all mines)
>> -1
{'id': 1, 'Serial Number': 'lasdkjf', 'x': 1, 'y': 1, 'armed': True, 'pin': ''}
Press enter to return to main menu
```

Code:

```
import requests
import json
import os
import server
import time

URL =
'https://coe892lab42026g07-fsa5fwhkdpf7b5bu.canadacentral-01.azurewebsites.net/
'
DELAY = 1

def displayMap():
    m = requests.get(URL + "map").json()
    for row in m["map"]:
        print("    " + str(row))

def clear():
    os.system('cls' if os.name == 'nt' else 'clear')

if __name__ == '__main__':
    while True:
        clear()
        print("")
        displayMap()
        print("")
        print("----OPERATOR----")
        print("")
        print("Spawn Rover: 1")
        print("See Rovers: 2")
        print("Edit Rover Commands: 3")
        print("Delete Rover: 4")
        print("Dispatch Rover: 5")
        print("Edit Map: 6")
        print("Create Mine: 7")
        print("Edit Mine: 8")
        print("See Mine Details: 9")
        print("Delete Mine: 10")
```

```
        print("Exit: -1")
        opIn = input(">> ")
        match opIn:
            case "-1":
                break
            case "1":
                clear()
                print("Spawn Rover")
                print("Commands: (L, R, M, D)")
                commands = input(">> ")
                body = server.createRoverReq(cmds=commands)
                r = requests.post(URL + "/rovers/", json=body.model_dump(),
allow_redirects=False).json()
                if r["Success"]:
                    print(f"Rover successfully spawned with ID {r["id"]}")
                print("Press enter to return to main menu")
                input()
            case "2":
                clear()
                print("See Rovers")
                print("Rover ID: (Enter -1 to see all rovers)")
                rID = input(">> ")
                if rID == "-1":
                    rovers = requests.get(URL + "rovers").json()
                    for r in rovers["rovers"]:
                        print(r)
                else:
                    rovers = requests.get(URL + f"rovers/{int(rID)}").json()
                    if rovers["Success"]:
                        print(rovers["rover"])
                    else:
                        print(rovers["error"])
                print("Press enter to return to main menu")
                input()
            case "3":
                clear()
                print("Update Rover Commands")
                print("Rover ID: ")
```

```
                rID = input(">> ")
                print("Commands: (L, R, M, D)")
                commands = input(">> ")
                body = server.updateCmdReq(cmds=commands)
                ret = requests.put(URL + f"rovers/{rID}/",
json=body.model_dump()).json()
                if ret["Success"]:
                    print("Commands updated")
                else:
                    print(ret["error"])
                print("Press enter to return to main menu")
                input()
            case "4":
                clear()
                print("Delete Rover")
                print("Rover ID: ")
                rID = input(">> ")
                ret = requests.delete(URL + f"rovers/{rID}").json()
                if ret["Success"]:
                    print(f"Rover {rID} successfully deleted")
                else:
                    print(ret["error"])
                print("Press enter to return to main menu")
                input()
            case "5":
                clear()
                print("Dispatch Rover")
                print("Rover ID: ")
                rID = input(">> ")
                ret = requests.post(URL + f"rovers/{rID}/dispatch/").json()
                if ret["Success"]:
                    print(f"Successfully dispatched Rover {rID}")
                    print(ret["rover"])
                else:
                    print(ret["error"])
                print("Press enter to return to main menu")
                input()
            case "6":
```

```
                clear()
                print("Edit Map")
                print("New Height: ")
                h = input(">> ")
                print("New Width: ")
                w = input(">> ")
                if w == "":
                    w = None
                if h == "":
                    h = None
                body = server.mapUpdateReq(height=h, width=w)
                ret = requests.put(URL + "map/", json=body.model_dump()).json()
                print("Press enter to return to main menu")
                input()
            case "7":
                clear()
                print("Create Mine")
                print("x position: ")
                x = input(">> ")
                print("y position: ")
                y = input(">> ")
                print("Serial Number: ")
                ser = input(">> ")
                body = server.createMineReq(x=int(x), y=int(y), serial=ser)
                ret = requests.post(URL + "mines/",
json=body.model_dump()).json()
                if ret["Success"]:
                    print(f"Mine successfully created at {x}, {y} with ID
{ret["id"]}")
                else:
                    print(ret["error"])
                print("Press enter to return to main menu")
                input()
            case "8":
                clear()
                print("Edit Mine")
                print("Mine ID: ")
                mID = input(">> ")
```

```
                print("x position: ")
                x = input(">> ")
                print("y position: ")
                y = input(">> ")
                print("Serial Number: ")
                ser = input(">> ")
                body = server.updateMineReq(x=None, y=None, serial=None,
armed=None, pin=None)
                if x == "":
                    body.x = None
                else:
                    body.x = int(x)
                if y == "":
                    body.y = None
                else:
                    body.y = int(y)
                if ser == "":
                    body.serial = None
                else:
                    body.serial = ser

                ret = requests.put(URL + f"mines/{mID}/",
json=body.model_dump()).json()
                if ret["Success"]:
                    print("Mine successfully edited:")
                    print(ret["mine"])
                else:
                    print(ret["error"])
                print("Press enter to return to main menu")
                input()
            case "9":
                clear()
                print("See Mine Details")
                print("Mine ID: (Enter -1 to see all mines)")
                mID = input(">> ")
                if mID == "-1":
                    ret = requests.get(URL + "mines").json()
                    for mine in ret["mines"]:
```

```
                        print(mine)
                else:
                    ret = requests.get(URL + f"mines/{mID}").json()
                    if ret["Success"]:
                        print(ret["mine"])
                    else:
                        print(ret["error"])
                print("Press enter to return to main menu")
                input()
            case "10":
                clear()
                print("Delete Mine")
                print("Mine ID: ")
                mID = input(">> ")
                ret = requests.delete(URL + f"mines/{mID}").json()
                if ret["Success"]:
                    print(f"Successfully deleted Mine {mID}")
                else:
                    print(ret["error"])
                print("Press enter to return to main menu")
                input()
```

Part 4

The server was containerized using the following Docker file:

```
# Use an official Python runtime as a base image
FROM python:3.12-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file (create one if it doesn't exist)
# Assuming you have a requirements.txt in Lab 4/ or create it with: fastapi,
uvicorn[standard], pydantic
COPY "requirements.txt" ./

# Install Python dependencies
```

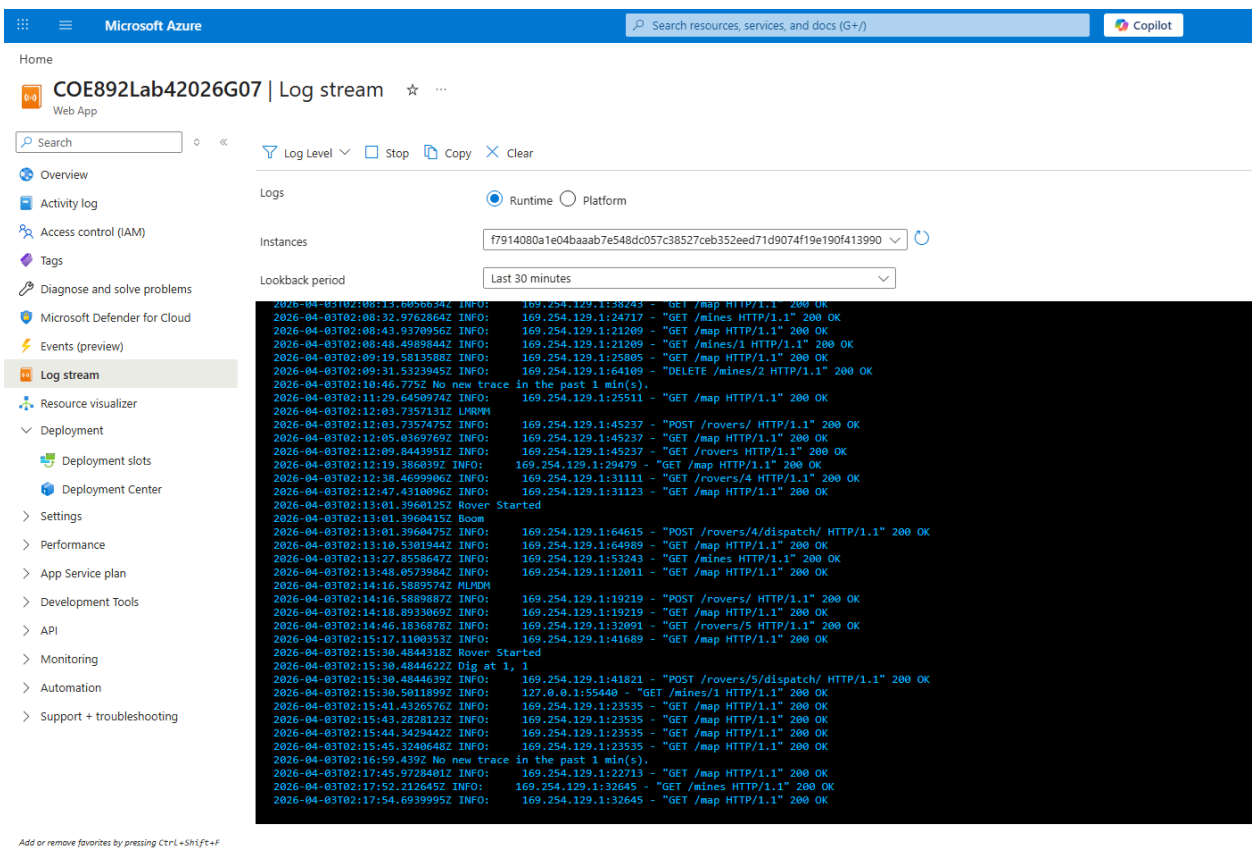
```
RUN pip install --no-cache-dir -r requirements.txt

# Copy the Lab 4 directory contents into the container
COPY "." "./Lab4"

# Expose the port the app runs on
EXPOSE 8000

# Command to run the application
CMD ["python", "Lab4/server.py"]
```

It was then pushed to Azure and run as a web app on this URL:
coe892lab42026g07-fsa5fwhkdpf7b5bu.canadacentral-01.azurewebsites.net



Conclusion

This lab successfully demonstrated an implementation of a RESTful API client-server setup using FastAPI and Python. The server was built using FastAPI to handle a defined set of HTTP requests, and the operator provided a user-friendly interface. The server was also deployed on Microsoft Azure, making it accessible to anyone.